Form & Field: Complete Project Guide

A file-by-file guide to the storefront, FastAPI backend, cart, and demo checkout
Prepared September 27, 2026

1. What This Project Is

Form & Field is a small, full-stack homewares storefront. It demonstrates a working product catalog, search and category filtering, a browser-side shopping bag, and a FastAPI JSON API. FastAPI serves both the website and the API, so the page and its API requests use the same origin.

The order button currently creates a **demo order only**. The app does not connect to Stripe, PayPal, or any other payment processor; it cannot charge a customer, store an order permanently, or collect customer or delivery details. A displayed order number is a sample receipt, not proof of payment or fulfillment.

2. At a Glance

Area	Technology	Responsibility
API and local web server	Python, FastAPI, Uvicorn	Serve the HTML, CSS, and JavaScript; expose product and checkout endpoints.
Request validation	Pydantic, through FastAPI	Parse JSON and reject invalid checkout shapes and quantities.
Browser interface	HTML5	Define the page, product grid, shopping-bag drawer, and accessible labels.
Browser behavior	Plain JavaScript (ES6+)	Load products; filter and sort; maintain the cart; call checkout.
Styling and responsiveness	Plain CSS	Define visual styling, layout, motion, and desktop/tablet/mobile breakpoints.
PDF guide generation	Python-Markdown, ReportLab	Convert this guide and generated source listings into the downloadable PDF. These are documentation-only dependencies.
Product photos and display fonts	Unsplash and Google Fonts	Supply externally hosted images and the DM Sans / Playfair Display fonts.
Payment	None	Checkout is simulated; no payment request is made.
Persistent database	None	Products and stock are Python in-memory data; the cart is browser local storage.

There is no React, Vue, npm package, JavaScript build step, database, ORM, login system, or payment SDK in this project.

3. Project Directory

```
shop_demo/
├── main.py                  FastAPI application, catalog, stock, API routes
├── requirements.txt         Python runtime dependency ranges
├── README.md                Short setup and API reference
├── .venv/                   Local Python environment; generated, not source
├── static/
│   ├── index.html           Storefront structure and text
│   ├── app.js               Browser state, rendering, cart, API calls
│   └── styles.css           Design system, layout, responsive styles
└── docs/
    ├── COMPLETE_GUIDE.md    This guide (the maintainable source document)
    ├── build_guide.py       Build the PDF and add current source appendices
    └── COMPLETE_GUIDE.pdf   Generated PDF copy of this guide
```

The Python virtual environment `.venv/` isolates this app's installed packages from system Python and from the rest of the workspace. It is not source code and should not be committed.

4. Requirements and Packages

4.1 Python version

Use Python **3.10 or later**. The current development environment uses Python 3.11.13. The app uses the modern built-in typing forms `list[CartLine]` and `list[dict]`; Python 3.10 satisfies the project's documented minimum.

4.2 Direct runtime dependencies

`requirements.txt` declares two packages:

- **FastAPI** (`fastapi>=0.115,<1.0`) provides the application class, HTTP route decorators, JSON serialization, error responses, automatic OpenAPI schema, and the `/docs` API explorer. The installed development version is 0.141.1.
- **Uvicorn** (`uvicorn>=0.30,<1.0`) is the ASGI server that listens on the local port and runs the FastAPI application. The installed development version is 0.54.0.

FastAPI brings in and uses **Starlette** for ASGI/web behavior and **Pydantic** for validation. They are transitive dependencies; they do not need separate lines in `requirements.txt`. The installed Pydantic development version is 2.13.5. Exact versions can vary because `requirements.txt` specifies compatible version ranges, not a fully pinned lockfile.

4.3 Python standard library

`main.py` also uses standard-library modules:

- `pathlib.Path` finds `static/` relative to `main.py`, even if the process starts in a different directory.
- `collections.Counter` combines repeated product IDs in a checkout request before stock and total calculations.
- `threading.Lock` prevents two requests served by this one Python process from simultaneously passing the same stock check and decrement.
- `uuid.uuid4` makes a short, hard-to-predict-looking sample order reference. The reference is not recorded in a database.

4.4 Browser dependencies

The browser needs no package installation. The site is HTML, CSS, and plain JavaScript delivered directly by FastAPI. The `Intl.NumberFormat`, `fetch`, `localStorage`, DOM APIs, and JavaScript language features used by the app are supplied by the browser.

The DM Sans and Playfair Display fonts load from Google Fonts. Home and product photos load from Unsplash. Those assets need network access; if the font request fails, CSS falls back to Georgia/Trebuchet MS, and if an image request fails, the image area retains its background color.

5. Install and Run

Run these commands from the `shop_demo` directory. On Linux/macOS with Bash or Zsh:

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
python -m uvicorn main:app --reload
```

For the C shell terminal shown in this workspace, activate the same environment with:

```
source .venv/bin/activate.csh
python -m pip install -r requirements.txt
python -m uvicorn main:app --reload
```

The `main:app` argument means “import the Python module `main.py` and run the FastAPI object named `app`.” Uvicorn's working directory should be `shop_demo`, because `main` is imported as a module from that directory. `--reload` restarts the development server after Python changes; it is a development convenience, not a production setting.

By default the site is at http://127.0.0.1:8000/. The automatically generated, interactive API reference is at http://127.0.0.1:8000/docs; the raw OpenAPI schema is at http://127.0.0.1:8000/openapi.json. If port 8000 is occupied, choose another available port with `--port 8001`, then use that port in the browser.

To leave the environment, use `deactivate` in Bash/Zsh, or `deactivate` in a C shell terminal after activation.

6. How the Pieces Fit Together

```
Browser: GET /
    -> Uvicorn runs FastAPI
    -> FastAPI returns static/index.html
Browser: GET /static/styles.css and /static/app.js
    -> FastAPI's StaticFiles serves the frontend assets
Browser: GET /api/products
    -> FastAPI returns the product list and current in-memory stock as JSON
User edits the bag
    -> JavaScript stores product IDs and quantities in localStorage
User presses "Place demo order"
    -> JavaScript POSTs item IDs and quantities to /api/checkout
    -> FastAPI validates, looks up current server prices, checks stock, totals,
       and decrements in-memory stock under a process-local lock
    -> FastAPI returns a sample JSON receipt
    -> JavaScript clears localStorage's cart and displays the receipt
```

The browser never sends an amount as an input to `/api/checkout`: it sends only product IDs and quantities. The browser calculates a convenient preview subtotal, but the server calculates the order total independently from its catalog. This is important because browser data can be edited by the shopper and must not set a chargeable price.

Because the browser and API are both served from `127.0.0.1:8000`, `fetch('/api/...')` is a **same-origin** request. The app does not need CORS configuration for its own page. If a separate frontend is later hosted on a different origin, the API's allowed origins must be configured deliberately.

7. Backend: `main.py`

7.1 App setup and paths

`BASE_DIR = Path(__file__).resolve().parent` identifies the folder containing `main.py`. `FastAPI(title=..., version=...)` constructs the ASGI application. `app.mount('/static', StaticFiles(...), name='static')` registers a static-file sub-application: the browser's `/static/...` URLs map to the project's `static/` folder.

The `GET /` route returns `FileResponse(BASE_DIR / 'static' / 'index.html')`. This makes the same FastAPI server responsible for the storefront HTML as well as its API.

7.2 Catalog and money representation

`PRODUCTS` is a Python list of eight dictionaries. Each product has an ID, name, category, integer price, color/variant text, optional badge, image URL, and image alternative text. Prices are **integer cents**: `12800` represents `$128.00`. Keeping money as integer minor units avoids floating-point rounding in price arithmetic.

`PRODUCT_BY_ID` is built once as a dictionary for efficient ID lookups. `STOCK` is a separate dictionary initialized to quantity 12 for each product. These are demonstration data; neither catalog edits nor stock changes survive a Python process restart. The same Unsplash photo URL is used by both lamp products.

7.3 Checkout request models

`CartLine` is a Pydantic model with `product_id: str` and `quantity: int`. `Field(gt=0, le=20)` requires a positive integer no greater than 20 on each input line. No price field exists in the model. Pydantic/FastAPI returns HTTP 422 when the JSON is malformed or fails these constraints.

`CheckoutRequest` wraps a required list named `items`. `Field(min_length=1, max_length=30)` requires at least one and at most 30 cart lines. Lines may repeat product IDs; the route merges these quantities before evaluating the final quantity. It separately rejects an aggregated quantity over 20 for one product.

An example request:

```
POST /api/checkout
Content-Type: application/json

{
  "items": [
    { "product_id": "arc-lamp", "quantity": 1 },
    { "product_id": "daily-cup", "quantity": 2 }
  ]
}
```

7.4 Product route

`GET /api/products` returns each product dictionary with a `stock` value added from `STOCK`. FastAPI converts this returned Python list/dictionary into JSON automatically. The frontend initially calls this endpoint to get product descriptions, cents-based prices, images, badges, and stock.

7.5 Demo checkout route, step by step

`@app.post('/api/checkout')` registers the function as an HTTP POST route. FastAPI parses the JSON request body into a `CheckoutRequest` instance before calling the function. A validation failure becomes a standard FastAPI 422 response without entering the function.

Within `checkout()`:

1. `Counter()` sums quantities by `product_id`, so duplicate lines cannot bypass the aggregate quantity check.
2. An unknown ID produces HTTP 404.
3. An aggregate quantity greater than 20 produces HTTP 422.
4. `CHECKOUT_LOCK` makes the stock-check-and-decrement block mutually exclusive between threads in this Python process. Insufficient stock produces HTTP 409.
5. The total is the sum of each server-owned catalog price (in cents) multiplied by quantity. Client-supplied prices are not part of the request model and cannot set this total.
6. The route subtracts the purchased quantities from the in-memory `STOCK` dictionary.
7. On success, it returns a generated `order_id`, `item_count`, integer-cent `total`, `currency`, and a demo confirmation `message`.

For the example above, using the current catalog, the server returns a total of 18,400 cents (`$184.00`) and an item count of 3. The exact generated order ID varies.

Example response:

```
{
  "order_id": "FF-XXXXXXXX",
  "item_count": 3,
  "total": 18400,
  "currency": "USD",
  "message": "Your order is confirmed. Thanks for shopping small."
}
```

The placeholder ID above is illustrative. Actual IDs are eight uppercase hexadecimal characters after the `FF-` prefix. The endpoint does **not** save that response anywhere and does **not** call a payment service.

7.6 HTTP outcomes

Situation	HTTP status	Where it comes from
Product list loaded	200	`GET /api/products`
Demo receipt created, stock decremented	200	Successful `POST /api/checkout`
Unknown product ID	404	Explicit `HTTPException` in checkout
Inventory less than requested	409	Explicit `HTTPException` inside locked checkout section
Invalid JSON/model; absent or empty `items`; quantities outside 1–20; over 30 input lines	422	FastAPI/Pydantic request validation
Repeated lines add up to more than 20 units for one product	422	Explicit aggregate check in checkout

7.7 Concurrency boundary

The `threading.Lock` is useful only inside the **one Python process** holding that lock. It does not coordinate other worker processes, other servers, or a database. Running multiple Uvicorn workers with this in-memory stock can therefore result in inconsistent availability. A production inventory update belongs in a persistent database transaction with suitable concurrency control.

8. Frontend: `static/index.html`

The HTML is the document structure; it does not contain the product catalog itself. It links the stylesheet at `/static/styles.css` and loads `/static/app.js` with `defer`. `defer` lets the browser parse the page before executing the script, so JavaScript can safely find the existing controls.

The page contains:

- An announcement bar, brand/header, category navigation, search field, and bag button.
- A hero section with a remote living-room image and a link to the product section.
- A shop section containing empty elements (`#category-filters` and `#product-grid`) that JavaScript fills from `/api/products`; sort and empty-state controls are also declared here.
- A short story/brand section and footer.
- A bag overlay and side drawer. Its product rows, count, and subtotal are populated dynamically. The checkout button is labelled **Place demo order** to indicate it is not a card-payment flow.

Semantic landmarks (`header`, `nav`, `main`, `section`, `footer`), headings, image `alt` text, control labels, an `aria-live` product grid, status messages, and an accessible dialog label help assistive technologies. CSS focus-visible styles make keyboard focus visible. When the bag is opened, JavaScript toggles dialog visibility/state and puts focus on the close button; Escape and clicking the backdrop close it.

Google Fonts is requested in the HTML `<head>`. CSS has a matching font import; the fonts are external, not vendored into the project. The hero and product image URLs also reference Unsplash; those are not bundled project assets.

9. Frontend Behavior: `static/app.js`

9.1 State and initialization

At the top of the script, `document.querySelector()` captures references to the product grid, search, filter area, sort control, drawer, bag rows, and checkout controls. `products` holds the current server response. `selectedCategory` starts at `All pieces`; `cart` is loaded from browser `localStorage` key `form-field-cart`.

The `init()` function fetches `GET /api/products`, then renders filters, product cards, and the bag and clears the busy state. If the fetch fails, it displays an unavailable message. `loadCart()` handles absent, invalid JSON, or an unexpected array; it falls back to an empty object. `saveCart()` persists changes in that browser's local storage.

Local storage is not shared across browsers or devices and can be edited by its owner.

9.2 Browse the products

`renderFilters()` creates category buttons using categories found in the API response. Clicking a filter changes the selected category and redraws filters and products. The header category links do the same and scroll to the shop section.

`visibleProducts()` narrows by selected category and by a case-insensitive search over product name, category, and color. Its sort options are featured/server order, ascending price, descending price, and alphabetical name. `renderProducts()` writes the resulting cards into the grid, formats cents as USD with `Intl.NumberFormat`, uses the server stock to disable sold-out products, and adds staggered appearance delays. Product images use lazy loading.

9.3 Add and edit bag items

The document-level click handler uses `event.target.closest()` to handle dynamically rendered Add-to-bag and quantity buttons. An add increments `cart[product.id]` if it is below the last loaded stock value, saves the cart, redraws the bag, and shows a short toast. The quantity minus button decrements; zero removes the item. The plus button is disabled at the known stock limit.

`cartEntries()` excludes IDs not present in the current product list and requires a positive integer quantity. `renderBag()` uses those entries to update the badge, drawer count, preview subtotal, product rows, and checkout-button enabled state. The browser subtotal is only a display estimate: the backend independently checks current stock and calculates its own total.

9.4 Place the demo order

When the checkout button is clicked, the handler disables the button, shows “Placing your order...”, and sends:

```
fetch('/api/checkout', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    items: cartEntries().map(([product_id, quantity]) => ({ product_id, quantity }))
  })
});
```

The request contains only IDs and quantities. On a successful HTTP response, JavaScript clears local storage's cart, reloads products (so the bag sees the reduced stock), and displays the order ID and server-returned total. On a failed HTTP response, it displays FastAPI's `detail` message and leaves the cart in place. Network errors display the JavaScript error message. The code stops there: it does not redirect to a payment page, request card details, create a payment intent, or wait for a payment webhook.

10. Styling: `static/styles.css`

The stylesheet defines a small color-and-type system using CSS custom properties: warm paper/white backgrounds, dark ink, muted labels, lime-green accent, and red-orange accent. Display headings use Playfair Display; interface text uses DM Sans, with local generic fallbacks. Product cards use consistent image aspect ratios and keep badges positioned independently from product text.

The desktop product grid has four columns. At widths up to 940 pixels, the header and hero rearrange and the grid has three columns. Up to 620 pixels, the hero becomes a vertical stack and the grid has two columns; the product filters remain horizontally usable. At 1500 pixels and above, content receives a maximum-width style of centered padding. The bag drawer has a viewport-relative height and fixed position so its item list can scroll independently.

Subtle card/image hover and entrance transitions are included. A `prefers-reduced-motion: reduce` rule reduces animation and smooth scrolling for people who request less motion. CSS focus-visible outlines identify keyboard focus. Layout width was checked in a 390-pixel browser viewport; the document did not overflow horizontally.

11. API Examples for Manual Testing

With Uvicorn running from the project directory:

```
curl -i http://127.0.0.1:8000/
```

```
curl -s http://127.0.0.1:8000/api/products
curl -i -X POST http://127.0.0.1:8000/api/checkout \
  -H 'Content-Type: application/json' \
  -d '{"items":[{"product_id":"daily-cup","quantity":2}]}'
```

The final command creates a **demo order** and reduces `daily-cup` in-memory stock by two. Repeat it only if that demonstration side effect is acceptable. An unknown ID returns 404; quantity above stock returns 409; invalid model data returns 422. The OpenAPI `/docs` page can also send these requests interactively.

12. Verification Performed for This Project

The working implementation was checked with:

- Python compilation (`py_compile main.py`).
- Browser-script parsing (`node --check static/app.js`).
- HTTP checks for the home page, stylesheet, eight-product catalog, successful checkout, and reduced stock.
- A checkout request containing an extra, incorrect client `price` value; the server ignored it and used the catalog price.
- Error checks for unknown products (404) and orders above remaining stock (409).
- Browser interactions for category filtering, search, adding a product, bag subtotal, and demo checkout confirmation.
- A 390-pixel viewport horizontal-overflow check and a successful hero-image load.

These are development checks, not a comprehensive automated test suite. No dedicated pytest suite is currently included.

13. Important Limitations and Production Work

This is an educational/local demo, not a live commerce system:

1. **No payment provider:** there is no Stripe/PayPal SDK, payment session, card-entry UI, authorization, capture, refund, or payment event handler. No money is collected.
2. **No durable order record:** an order ID and response exist only for that request; there is no order table or purchase-history endpoint.
3. **No durable catalog or stock:** products and stock reset when the Python process restarts. A process-local lock cannot coordinate multiple application processes.
4. **Stock is reduced before any payment:** the demo treats its receipt as immediately successful and subtracts inventory at that point. A production payment flow must not tell the user it was paid before confirmed payment. It needs a defined reservation/expiry/release strategy and transactional stock updates.
5. **No customer/shipping/tax data:** there is no address, customer identity, shipping-rate calculation, tax calculation, order fulfillment, email, or return workflow.
6. **Cart is browser-local:** it does not synchronize between browsers/devices and is not a server-side, authenticated cart. The API must still validate every submitted ID, price, quantity, stock count, and order state.
7. **No payment secrets or HTTPS configuration:** those become essential when deploying or calling a real payment provider. Private keys must live in server-side environment/secret configuration, never in JavaScript or committed files.
8. **Externally hosted assets:** fonts and images depend on third-party availability and their terms. Store/optimize licensed production assets locally if reliability, privacy, or licensing requirements call for it.

Recommended real-payment sequence

For a provider-hosted checkout such as Stripe Checkout, the safe high-level sequence is:

1. Browser posts the cart to this server, still sending only product IDs and quantities.
2. Server validates the cart, reads trusted current prices from the database, calculates delivery/tax where appropriate, and creates a **pending** order with a unique internal ID.
3. Server creates a provider Checkout Session for the pending order's calculated amount, using a secret API key held only by the server, and returns the provider's hosted checkout URL.
4. Browser redirects the shopper to that provider-hosted page. Raw card details go directly to the provider, not to this storefront server.
5. Provider redirects the shopper back to a result page. This redirect alone is not proof the customer paid.
6. Provider sends a server-to-server webhook. The app verifies the webhook signature and event and then updates the order to `paid` (or a failure/refund state) in a database. A verified webhook is the payment-state source of truth.
7. The app finalizes/reduces inventory transactionally and shows confirmation only when server-side payment status is verified. Retries use provider idempotency keys so they cannot accidentally create duplicate charges/orders.

The exact implementation depends on the chosen provider, test/live mode, shipping/tax rules, database, and deployment URL. The current app's order route would need to change rather than simply sending the existing demo response to a payment service.

14. Run and Update This Documentation

Edit `docs/COMPLETE_GUIDE.md` for the explanatory material. The PDF build script reads that document plus the current runtime source files and adds their full text as a source appendix; source listings therefore stay synchronized without copying code into this Markdown file manually.

Install the documentation-only dependencies once, then run the builder from `shop_demo`:

```
python -m pip install -r docs/requirements.txt
python docs/build_guide.py
```

The resulting PDF is `docs/COMPLETE_GUIDE.pdf`. `Markdown` converts the guide to an HTML structure, and `ReportLab` lays out the document and source code blocks. Neither documentation package is needed to run the storefront; they are not listed in the app's `requirements.txt`.

Appendix: Complete Source Files

These listings are generated directly from the current project files.

main.py

```
from collections import Counter
from pathlib import Path
from threading import Lock
from uuid import uuid4

from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, Field


BASE_DIR = Path(__file__).resolve().parent

PRODUCTS = [
    {
        "id": "arc-lamp",
        "name": "Arc Table Lamp",
        "category": "Lighting",
        "price": 12800,
        "color": "Chalk / butter",
        "badge": "BESTSELLER",
        "image": "https://images.unsplash.com/photo-1507473885765-
e6ed057f782c?auto=format&fit=crop&w=900&q=85",
        "alt": "Sculptural table lamp glowing on a side table",
    },
    {
        "id": "form-vase",
        "name": "Form Stoneware Vase",
        "category": "Objects",
        "price": 6400,
        "color": "Speckled white",
        "badge": "JUST IN",
        "image": "https://images.unsplash.com/photo-1578500494198-
246f612d3b3d?auto=format&fit=crop&w=900&q=85",
        "alt": "Minimal ceramic vase with a sculptural silhouette",
    },
    {
        "id": "linen-throw",
        "name": "Everyday Linen Throw",
        "category": "Textiles",
        "price": 11200,
        "color": "Rust / oat",
        "badge": "",
        "image": "https://images.unsplash.com/photo-1600210492486-
724fe5c67fb0?auto=format&fit=crop&w=900&q=85",
        "alt": "Soft throw draped over a contemporary armchair",
    },
    {
        "id": "orbit-chair",
        "name": "Orbit Lounge Chair",
        "category": "Furniture",
        "price": 38900,
        "color": "Forest green",
        "badge": "LOW STOCK",
        "image": "https://images.unsplash.com/photo-1567538096630-
e0c55bd6374c?auto=format&fit=crop&w=900&q=85",
        "alt": "Contemporary lounge chair in a light-filled room",
    },
    {
        "id": "daily-cup",
        "name": "Daily Ceramic Cup",
        "category": "Objects",
        "price": 2800,
        "color": "Moss glaze",
        "badge": "",
        "image": "https://images.unsplash.com/photo-1514228742587-
6b1558fcca3d?auto=format&fit=crop&w=900&q=85",
        "alt": "Handmade ceramic cup on a bright tabletop",
    },
    {
        "id": "fold-stool",
        "name": "Fold Side Table",
        "category": "Furniture",
        "price": 17600,
        "color": "Natural ash",
        "badge": "",
        "image": "https://images.unsplash.com/photo-1499933374294-
4584851497cc?auto=format&fit=crop&w=900&q=85",
        "alt": "Compact wood side table beside a reading chair",
    },
    {
        "id": "soft-glow",
        "name": "Soft Glow Pendant",
        "category": "Lighting",
        "price": 21400,
        "color": "Citrus / white",
```

```
        "badge": "JUST IN",
        "image": "https://images.unsplash.com/photo-1507473885765-
e6ed057f782c?auto=format&fit=crop&w=900&q=85",
        "alt": "Warm pendant light above a carefully set interior",
    },
    {
        "id": "grid-cushion",
        "name": "Grid Cotton Cushion",
        "category": "Textiles",
        "price": 5800,
        "color": "Poppy / cream",
        "badge": "",
        "image": "https://images.unsplash.com/photo-1584100936595-
c0654b55a2e2?auto=format&fit=crop&w=900&q=85",
        "alt": "Textured cushion adding a bright accent to a sofa",
    },
]

PRODUCT_BY_ID = {product["id"]: product for product in PRODUCTS}
STOCK = {product["id"]: 12 for product in PRODUCTS}
CHECKOUT_LOCK = Lock()

app = FastAPI(title="Form & Field Shop", version="1.0.0")
app.mount("/static", StaticFiles(directory=BASE_DIR / "static"), name="static")


class CartLine(BaseModel):
    product_id: str
    quantity: int = Field(gt=0, le=20)


class CheckoutRequest(BaseModel):
    items: list[CartLine] = Field(min_length=1, max_length=30)


@app.get("/")
def storefront():
    return FileResponse(BASE_DIR / "static" / "index.html")


@app.get("/api/products")
def list_products():
    return [dict(product, stock=STOCK[product["id"]]) for product in PRODUCTS]


@app.post("/api/checkout")
def checkout(request: CheckoutRequest):
    quantities = Counter()
    for line in request.items:
        quantities[line.product_id] += line.quantity

    if any(product_id not in PRODUCT_BY_ID for product_id in quantities):
        raise HTTPException(status_code=404, detail="One or more products were not found.")
    if any(quantity > 20 for quantity in quantities.values()):
        raise HTTPException(status_code=422, detail="A maximum of 20 of each item can be ordered.")

    with CHECKOUT_LOCK:
        if any(quantity > STOCK[product_id] for product_id, quantity in quantities.items()):
            raise HTTPException(status_code=409, detail="An item in your bag is no longer available in
that quantity.")

        total = sum(PRODUCT_BY_ID[product_id]["price"] * quantity for product_id, quantity in quantities.
items())
        for product_id, quantity in quantities.items():
            STOCK[product_id] -= quantity

    return {
        "order_id": "FF-" + uuid4().hex[:8].upper(),
        "item_count": sum(quantities.values()),
        "total": total,
        "currency": "USD",
        "message": "Your order is confirmed. Thanks for shopping small.",
    }
```

requirements.txt

```
fastapi>=0.115,<1.0
uvicorn>=0.30,<1.0
```

static/index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="theme-color" content="#fbfaf8">
  <meta name="description" content="Thoughtful things for the spaces you live in. Shop considered
homewares from Form & Field.">
  <title>Form & Field — Things to live with</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;650;
700&family=Playfair+Display:wght@500;600&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="/static/styles.css">
  <script src="/static/app.js" defer></script>
</head>
<body>
  <div class="announcement">A little something extra: free shipping on orders over $150</div>
  <header class="site-header">
    <a class="wordmark" href="#top" aria-label="Form and Field home">form<span>&</span>field<span
class="wordmark-period">.</span></a>
    <nav class="main-nav" aria-label="Main navigation">
      <a class="nav-link is-active" href="#shop">Shop all</a>
      <a class="nav-link" href="#shop" data-nav-category="Objects">Objects</a>
      <a class="nav-link" href="#shop" data-nav-category="Furniture">Furniture</a>
      <a class="nav-link" href="#story">Our point of view</a>
    </nav>
    <div class="header-actions">
      <label class="search-box">
        <svg aria-hidden="true" viewBox="0 0 20 20" fill="none"><circle cx="8.8" cy="8.8" r="5.8"></
circle><path d="m13 13 4 4"></path></svg>
        <input id="search-input" type="search" placeholder="Find your thing" aria-label="Search
products">
      </label>
      <button id="bag-open" class="bag-button" type="button" aria-label="Open shopping bag">Bag <span
id="bag-count">0</span></button>
    </div>
  </header>

  <main id="top">
    <section class="hero" aria-labelledby="hero-title">
      <div class="hero-copy">
        <p class="eyebrow"><span class="eyebrow-mark"></span> The considered home, since 2018</p>
        <h1 id="hero-title">Good things<br>to live <span>with.</span></h1>
        <p class="hero-description">Useful, lovely, and made to stay. Meet the everyday pieces that make
a space feel a little more like yours.</p>
        <a class="hero-link" href="#shop">Find your new favorite <span aria-hidden="true">&#8595;</
span></a>
        <div class="hero-note"><span class="note-spark">✳</span><span>Considered by us.<br>At home with
you.</span></div>
      </div>
      <div class="hero-image-wrap">
        <img class="hero-image" src="https://images.unsplash.com/photo-1600210492486-
724fe5c67fb0?auto=format&fit=crop&w=1600&q=90" alt="Sunlit, lived-in living room with thoughtful
furniture and natural textures" fetchpriority="high">
        <div class="image-caption"><span>THE SLOW LIVING EDIT</span><span>01 / 04</span></div>
        <div class="hero-stamp" aria-hidden="true"><span>MADE FOR</span><span>REAL</span><span>LIFE
&#8599;</span></div>
      </div>
      <div class="hero-footnote">Small-batch pieces, not big-box things <span>✳</span></div>
    </section>

    <section class="shop-section" id="shop" aria-labelledby="shop-title">
      <div class="shop-heading">
        <div>
          <p class="eyebrow section-eyebrow">KEEPERS, NOT CLUTTER</p>
          <h2 id="shop-title">The good stuff<span>.</span></h2>
        </div>
        <p class="shop-intro">Things made with care, chosen to be<br>used and loved every day.</p>
      </div>
      <div class="shop-toolbar">
        <div id="category-filters" class="category-filters" aria-label="Filter by category"></div>
        <label class="sort-control">Sort <select id="sort-select" aria-label="Sort products"><option
value="featured">Featured</option><option value="price-asc">Price: low to high</option><option
value="price-desc">Price: high to low</option><option value="name">Name</option></select></label>
      </div>
      <div id="product-grid" class="product-grid" aria-live="polite" aria-busy="true"></div>
      <p id="empty-state" class="empty-state" hidden>No pieces found. Try a different search.</p>
    </section>

    <section class="story-section" id="story" aria-label="Our approach">
      <p class="story-symbol" aria-hidden="true">✳</p>
      <p class="story-quote">Less, but <span>better loved.</span></p>
      <p class="story-copy">We work with independent makers to bring you honest materials, thoughtful
details, and things that feel better the longer you have them.</p>
      <a class="story-link" href="#shop">A little more about us <span aria-hidden="true">&#8594;</span></
```

```
a>
    </section>
  </main>

  <footer class="site-footer"><a class="wordmark footer-wordmark" href="#top">form<span>&</
span>field<span class="wordmark-period">.</span></a><p>Made for the everyday. Designed to last.</
p><span>&copy; Form & Field 2026</span></footer>

  <div id="bag-overlay" class="bag-overlay" hidden></div>
  <aside id="bag-drawer" class="bag-drawer" role="dialog" aria-modal="true" aria-labelledby="bag-title"
aria-hidden="true" inert>
    <div class="drawer-header"><div><p class="eyebrow section-eyebrow">YOUR GOOD THINGS</p><h2 id="bag-
title">Your bag <span id="drawer-count">(0)</span></h2></div><button id="bag-close" class="icon-button"
type="button" aria-label="Close bag">&#215;</button></div>
    <div id="bag-items" class="bag-items"></div>
    <div class="drawer-footer"><div class="subtotal-row"><span>Subtotal</span><strong id="bag-
subtotal">$0.00</strong></div><p class="shipping-note">Shipping & taxes calculated at checkout.</
p><button id="checkout-button" class="checkout-button" type="button" disabled>Place demo order
<span>&#8599;</span></button><p id="checkout-message" class="checkout-message" role="status"></p></div>
  </aside>
  <div id="toast" class="toast" role="status" aria-live="polite"></div>
</body>
</html>
```

static/styles.css

```
@import url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;650;
700&family=Playfair+Display:wght@500;600&display=swap');

:root {
  color-scheme: light;
  --paper: #fbfaf8;
  --white: #fff;
  --ink: #262923;
  --muted: #77786e;
  --line: #e9e7e1;
  --green: #d7ef64;
  --red: #dc5d3e;
  --serif: 'Playfair Display', Georgia, serif;
  --sans: 'DM Sans', 'Trebuchet MS', sans-serif;
  font-family: var(--sans);
  color: var(--ink);
  background: var(--paper);
  font-synthesis: none;
  text-rendering: optimizeLegibility;
}

* { box-sizing: border-box; }
html { scroll-behavior: smooth; }
body { min-width: 360px; margin: 0; }
button, input, select { font: inherit; }
button, a, select { -webkit-tap-highlight-color: transparent; }
a { color: inherit; text-decoration: none; }
button:focus-visible, a:focus-visible, input:focus-visible, select:focus-visible { outline: 2px solid
var(--red); outline-offset: 4px; }
[hidden] { display: none !important; }

.announcement { display: grid; min-height: 32px; place-items: center; background: var(--green); font-
size: 11px; font-weight: 600; }
.site-header { position: relative; z-index: 2; display: grid; min-height: 77px; grid-template-columns:
1fr 1.55fr 1fr; align-items: center; padding: 0 6.4%; border-bottom: 1px solid var(--line); background:
var(--paper); }
.wordmark { width: max-content; font-size: 23px; font-weight: 700; line-height: 1; letter-spacing: -.9px;
 }
.wordmark > span:first-child { color: var(--red); }
.wordmark-period { color: var(--red); }
.main-nav, .header-actions { display: flex; align-items: center; }
.main-nav { justify-content: center; gap: 32px; }
.nav-link { position: relative; padding: 9px 0; color: #66675f; font-size: 12px; transition: color .2s; }
.nav-link:hover, .nav-link.is-active { color: var(--ink); }
.nav-link.is-active::after { position: absolute; right: 0; bottom: 2px; left: 0; height: 1px; background:
 var(--red); content: ''; }
.header-actions { justify-content: flex-end; gap: 22px; }
.search-box { display: flex; width: 156px; align-items: center; gap: 8px; border-bottom: 1px solid var(--
line); }
.search-box svg { width: 16px; flex: 0 0 16px; stroke: var(--muted); stroke-width: 1.5; }
.search-box input { width: 100%; padding: 7px 0; border: 0; outline: none; background: transparent; font-
size: 11px; }
.search-box input:focus-visible { outline: none; }
.bag-button { display: flex; min-width: 63px; align-items: center; justify-content: flex-end; gap: 8px;
border: 0; background: none; color: var(--ink); cursor: pointer; font-size: 12px; }
.bag-button span { display: grid; width: 22px; aspect-ratio: 1; place-items: center; border-radius: 50%;
background: var(--ink); color: white; font-size: 10px; }

.hero { position: relative; display: grid; min-height: 510px; grid-template-columns: 45.7% 54.3%;
padding: 38px 6.4% 54px; }
.hero-copy { position: relative; z-index: 1; display: flex; flex-direction: column; align-items: flex-
start; justify-content: center; padding: 0 18px 18px 0; }
.eyebrow { display: flex; align-items: center; gap: 9px; margin: 0; color: var(--muted); font-size: 9px;
font-weight: 600; }
.eyebrow-mark { width: 7px; height: 7px; border-radius: 50%; background: var(--red); }
.hero h1, .shop-heading h2, .drawer-header h2 { margin: 0; font-weight: 500; }
.hero h1 { margin-top: 23px; font-family: var(--serif); font-size: clamp(52px, 5.35vw, 77px); line-
height: 1.075; }
.hero h1 span { color: var(--red); font-style: italic; }
.hero-description { max-width: 345px; margin: 19px 0 0; color: var(--muted); font-size: 13px; line-
height: 1.85; }
.hero-link { display: flex; align-items: center; gap: 20px; margin-top: 25px; border-bottom: 1px solid
var(--ink); padding-bottom: 6px; font-size: 11px; font-weight: 600; }
.hero-link span { color: var(--red); font-size: 16px; transition: transform .2s; }
.hero-link:hover span { transform: translateY(3px); }
.hero-note { display: flex; align-items: center; gap: 10px; margin-top: 34px; color: var(--muted); font-
size: 9px; line-height: 1.6; }
.note-spark { color: var(--red); font-size: 18px; }
.hero-image-wrap { position: relative; min-height: 404px; overflow: hidden; background: #e8e9e3; }
.hero-image { position: absolute; width: 100%; height: 100%; object-fit: cover; object-position: 50% 54%;
 }
.image-caption { position: absolute; right: 15px; bottom: 14px; left: 15px; display: flex; justify-
content: space-between; color: white; font-size: 8px; font-weight: 600; text-shadow: 0 1px 6px #15161088;
 }
.hero-stamp { position: absolute; top: 19px; right: 19px; display: grid; width: 74px; aspect-ratio: 1;
align-content: center; justify-items: center; border-radius: 50%; background: var(--green); font-size:
```

```
9px; font-weight: 600; line-height: 1.5; transform: rotate(9deg); }
.hero-stamp span:nth-child(2) { font-family: var(--serif); font-size: 17px; }
.hero-footnote { position: absolute; right: 6.4%; bottom: 16px; display: flex; gap: 8px; color: var(--
muted); font-size: 9px; }
.hero-footnote span { color: var(--red); }

.shop-section { padding: 70px 6.4% 92px; background: var(--white); }
.shop-heading { display: flex; align-items: flex-end; justify-content: space-between; }
.section-eyebrow { color: var(--red); font-size: 9px; }
.shop-heading h2 { margin-top: 8px; font-family: var(--serif); font-size: 38px; line-height: 1.2; }
.shop-heading h2 span { color: var(--red); }
.shop-intro { margin: 0 0 4px; color: var(--muted); font-size: 11px; line-height: 1.8; }
.shop-toolbar { display: flex; min-height: 58px; align-items: center; justify-content: space-between;
margin-top: 27px; border-top: 1px solid var(--line); border-bottom: 1px solid var(--line); }
.category-filters { display: flex; flex-wrap: wrap; align-items: center; gap: 6px; }
.filter-button { border: 0; border-radius: 2px; padding: 8px 12px; background: transparent; color: var(--
muted); cursor: pointer; font-size: 10px; transition: color .18s, background .18s; }
.filter-button:hover { color: var(--ink); }
.filter-button.is-active { background: var(--ink); color: white; }
.sort-control { display: flex; align-items: center; gap: 6px; color: var(--muted); font-size: 10px; }
.sort-control select { max-width: 110px; border: 0; padding: 6px 3px; background: transparent; color:
var(--ink); cursor: pointer; font-size: 10px; }
.product-grid { display: grid; grid-template-columns: repeat(4, minmax(0, 1fr)); gap: 35px 19px; padding-
top: 23px; }
.product-card { min-width: 0; animation: appear .36s both; }
.product-image-wrap { position: relative; overflow: hidden; aspect-ratio: 4 / 4.65; background: #f0efeb;
}
.product-image { width: 100%; height: 100%; object-fit: cover; transition: transform .48s cubic-bezier(.
2, .65, .3, 1); }
.product-card:hover .product-image { transform: scale(1.045); }
.product-badge { position: absolute; top: 10px; left: 10px; max-width: calc(100% - 20px); padding: 6px
8px; background: var(--white); font-size: 8px; font-weight: 600; }
.product-info { padding-top: 12px; }
.product-title-row { display: flex; justify-content: space-between; gap: 8px; }
.product-name { min-width: 0; margin: 0; overflow-wrap: anywhere; font-size: 12px; font-weight: 500; }
.product-price { flex: 0 0 auto; margin: 0; font-size: 12px; font-weight: 600; }
.product-color { margin: 5px 0 0; color: var(--muted); font-size: 9px; }
.add-button { width: 100%; min-height: 34px; margin-top: 12px; border: 1px solid var(--line); background:
 transparent; color: var(--ink); cursor: pointer; font-size: 10px; transition: border-color .18s,
background .18s, color .18s; }
.add-button:hover:not(:disabled) { border-color: var(--ink); background: var(--ink); color: var(--white);
 }
.add-button:disabled { color: var(--muted); cursor: not-allowed; }
.empty-state { padding: 50px 0; color: var(--muted); text-align: center; font-size: 13px; }

.story-section { display: flex; min-height: 330px; flex-direction: column; align-items: center; justify-
content: center; padding: 54px 20px; text-align: center; }
.story-symbol { margin: 0 0 13px; color: var(--red); font-size: 20px; }
.story-quote { margin: 0; font-family: var(--serif); font-size: 35px; }
.story-quote span { color: var(--red); font-style: italic; }
.story-copy { max-width: 420px; margin: 15px 0 0; color: var(--muted); font-size: 11px; line-height: 1.8;
 }
.story-link { display: flex; align-items: center; gap: 9px; margin-top: 18px; padding-bottom: 4px;
border-bottom: 1px solid var(--line); font-size: 10px; font-weight: 600; }
.story-link span { color: var(--red); }
.site-footer { display: flex; min-height: 79px; align-items: center; justify-content: space-between; gap:
 16px; border-top: 1px solid var(--line); padding: 15px 6.4%; }
.footer-wordmark { font-size: 18px; }
.site-footer p, .site-footer > span { color: var(--muted); font-size: 9px; }

.bag-overlay { position: fixed; z-index: 5; inset: 0; background: #1d201b70; animation: fade-in .18s; }
.bag-drawer { position: fixed; z-index: 6; top: 0; right: 0; display: flex; width: min(410px, 100%);
height: 100dvh; flex-direction: column; padding: 27px 27px 22px; background: var(--paper); box-shadow: -
14px 0 40px #1112; transform: translateX(103%); transition: transform .26s cubic-bezier(.2, .7, .2, 1); }
.bag-drawer.is-open { transform: translateX(0); }
.drawer-header { display: flex; align-items: flex-start; justify-content: space-between; padding-bottom:
21px; border-bottom: 1px solid var(--line); }
.drawer-header h2 { margin-top: 8px; font-family: var(--serif); font-size: 29px; }
.drawer-header h2 span { color: var(--muted); font-family: var(--sans); font-size: 12px; }
.icon-button { display: grid; width: 34px; aspect-ratio: 1; place-items: center; border: 1px solid var(--
line); background: transparent; cursor: pointer; font-size: 22px; line-height: 1; }
.bag-items { flex: 1; overflow-y: auto; }
.empty-bag { padding: 30px 0; color: var(--muted); font-size: 12px; }
.bag-line { display: grid; grid-template-columns: 77px 1fr auto; align-items: start; gap: 13px; padding:
17px 0; border-bottom: 1px solid var(--line); }
.bag-line img { width: 77px; aspect-ratio: 1; object-fit: cover; background: #f0efeb; }
.bag-line-info h3 { margin: 2px 0 5px; font-size: 11px; font-weight: 500; }
.bag-line-info p { margin: 0; color: var(--muted); font-size: 9px; }
.quantity-control { display: flex; width: max-content; align-items: center; gap: 12px; margin-top: 12px;
border: 1px solid var(--line); padding: 4px 7px; }
.quantity-control button { width: 16px; border: 0; background: transparent; cursor: pointer; font-size:
12px; }
.quantity-control span { min-width: 9px; text-align: center; font-size: 10px; }
.bag-line-price { margin: 2px 0 0; font-size: 10px; font-weight: 600; }
.drawer-footer { padding-top: 17px; border-top: 1px solid var(--line); }
.subtotal-row { display: flex; justify-content: space-between; font-size: 13px; }
.shipping-note { margin: 8px 0 15px; color: var(--muted); font-size: 9px; }
.checkout-button { display: flex; width: 100%; min-height: 46px; align-items: center; justify-content:
space-between; border: 0; padding: 0 14px; background: var(--ink); color: white; cursor: pointer; font-
```

```
size: 11px; transition: background .18s; }
.checkout-button:hover:not(:disabled) { background: #44473d; }
.checkout-button:disabled { background: #b9bab3; cursor: not-allowed; }
.checkout-button span { color: var(--green); font-size: 15px; }
.checkout-message { min-height: 14px; margin: 10px 0 0; color: #397243; font-size: 10px; line-height: 1.
6; }
.checkout-message.is-error { color: var(--red); }
.toast { position: fixed; z-index: 8; right: 22px; bottom: 22px; padding: 12px 16px; background: var(--
ink); color: white; opacity: 0; pointer-events: none; font-size: 11px; transform: translateY(8px);
transition: opacity .2s, transform .2s; }
.toast.is-visible { opacity: 1; transform: translateY(0); }

@keyframes appear { from { opacity: 0; transform: translateY(8px); } to { opacity: 1; transform:
translateY(0); } }
@keyframes fade-in { from { opacity: 0; } to { opacity: 1; } }

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { scroll-behavior: auto !important; animation-duration: .01ms !important;
animation-iteration-count: 1 !important; transition-duration: .01ms !important; }
}
@media (min-width: 1500px) { .site-header, .hero, .shop-section, .site-footer { padding-right: max(6.4%,
calc((100% - 1320px) / 2)); padding-left: max(6.4%, calc((100% - 1320px) / 2)); } }
@media (max-width: 940px) {
  .site-header { grid-template-columns: 1fr 1fr; padding-right: 4%; padding-left: 4%; }
  .main-nav { grid-column: 1 / -1; grid-row: 2; justify-content: flex-start; gap: 24px; padding: 0 0
11px; }
  .header-actions { grid-column: 2; grid-row: 1; }
  .hero { min-height: 450px; grid-template-columns: 45% 55%; padding-right: 4%; padding-left: 4%; }
  .hero h1 { font-size: 56px; }
  .hero-footnote { right: 4%; }
  .shop-section { padding-right: 4%; padding-left: 4%; }
  .product-grid { grid-template-columns: repeat(3, minmax(0, 1fr)); }
}
@media (max-width: 620px) {
  body { min-width: 320px; }
  .announcement { min-height: 29px; font-size: 9px; }
  .site-header { min-height: 66px; }
  .wordmark { font-size: 20px; }
  .main-nav { gap: 19px; overflow-x: auto; scrollbar-width: none; }
  .main-nav::-webkit-scrollbar { display: none; }
  .nav-link { flex: 0 0 auto; font-size: 10px; }
  .search-box { width: 108px; }
  .search-box input { font-size: 10px; }
  .header-actions { gap: 9px; }
  .bag-button { min-width: auto; gap: 6px; font-size: 10px; }
  .hero { min-height: unset; grid-template-columns: 1fr; padding: 38px 5% 46px; }
  .hero-copy { padding: 0 0 24px; }
  .hero h1 { margin-top: 19px; font-size: 60px; }
  .hero-description { max-width: 310px; margin-top: 14px; font-size: 12px; }
  .hero-link { margin-top: 17px; }
  .hero-note { display: none; }
  .hero-image-wrap { min-height: 250px; aspect-ratio: 1.27; }
  .hero-stamp { top: 13px; right: 13px; width: 64px; }
  .hero-footnote { right: 5%; bottom: 15px; font-size: 8px; }
  .shop-section { padding: 48px 5% 61px; }
  .shop-heading { align-items: flex-start; flex-direction: column; gap: 8px; }
  .shop-heading h2 { font-size: 33px; }
  .shop-intro br { display: none; }
  .shop-toolbar { align-items: flex-start; flex-direction: column; gap: 8px; margin-top: 18px; padding:
10px 0; }
  .category-filters { gap: 0; }
  .filter-button { padding: 8px 10px; font-size: 9px; }
  .sort-control { align-self: flex-end; }
  .product-grid { grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 26px 12px; padding-top: 17px; }
  .product-image-wrap { aspect-ratio: 4 / 4.8; }
  .product-badge { top: 7px; left: 7px; padding: 5px 6px; font-size: 7px; }
  .product-name, .product-price { font-size: 10px; }
  .product-color { font-size: 8px; }
  .add-button { min-height: 32px; margin-top: 9px; font-size: 9px; }
  .story-section { min-height: 290px; }
  .story-quote { font-size: 30px; }
  .site-footer { align-items: flex-start; flex-direction: column; gap: 7px; padding: 21px 5%; }
  .site-footer p { margin: 0; }
  .bag-drawer { padding-right: 20px; padding-left: 20px; }
}
```

static/app.js

```
const productGrid = document.querySelector('#product-grid');
const categoryFilters = document.querySelector('#category-filters');
const searchInput = document.querySelector('#search-input');
const sortSelect = document.querySelector('#sort-select');
const emptyState = document.querySelector('#empty-state');
const bagDrawer = document.querySelector('#bag-drawer');
const bagOverlay = document.querySelector('#bag-overlay');
const bagItems = document.querySelector('#bag-items');
const checkoutButton = document.querySelector('#checkout-button');
const checkoutMessage = document.querySelector('#checkout-message');

const currency = new Intl.NumberFormat('en-US', { style: 'currency', currency: 'USD' });
let products = [];
let selectedCategory = 'All pieces';
let toastTimer;
let cart = loadCart();

function loadCart() {
  try {
    const saved = JSON.parse(localStorage.getItem('form-field-cart') || '{}');
    return saved && typeof saved === 'object' && !Array.isArray(saved) ? saved : {};
  } catch {
    return {};
  }
}

function saveCart() {
  localStorage.setItem('form-field-cart', JSON.stringify(cart));
}

function getProduct(id) {
  return products.find((product) => product.id === id);
}

function renderFilters() {
  const categories = ['All pieces', ...new Set(products.map((product) => product.category))];
  categoryFilters.innerHTML = '';
  categories.forEach((category) => {
    const button = document.createElement('button');
    button.type = 'button';
    button.className = `filter-button${selectedCategory === category ? ' is-active' : ''}`;
    button.textContent = category;
    button.setAttribute('aria-pressed', String(selectedCategory === category));
    button.addEventListener('click', () => {
      selectedCategory = category;
      document.querySelectorAll('[data-nav-category]').forEach((link) => link.classList.remove('is-
active'));
      renderFilters();
      renderProducts();
    });
    categoryFilters.append(button);
  });
}

function visibleProducts() {
  const query = searchInput.value.trim().toLowerCase();
  const filtered = products.filter((product) => {
    const matchesCategory = selectedCategory === 'All pieces' || product.category === selectedCategory;
    const matchesSearch = `${product.name} ${product.category} ${product.color}`.toLowerCase().includes(
query);
    return matchesCategory && matchesSearch;
  });

  switch (sortSelect.value) {
    case 'price-asc': return filtered.sort((a, b) => a.price - b.price);
    case 'price-desc': return filtered.sort((a, b) => b.price - a.price);
    case 'name': return filtered.sort((a, b) => a.name.localeCompare(b.name));
    default: return filtered;
  }
}

function renderProducts() {
  const shown = visibleProducts();
  emptyState.hidden = shown.length > 0;
  productGrid.innerHTML = shown.map((product, index) => `
    <article class="product-card" style="animation-delay:${Math.min(index * 35, 245)}ms">
      <div class="product-image-wrap">
        <img class="product-image" src="${product.image}" alt="${product.alt}" loading="lazy">
        ${product.badge ? `<span class="product-badge">${product.badge}</span>` : ''}
      </div>
      <div class="product-info">
        <div class="product-title-row"><h3 class="product-name">${product.name}</h3><p class="product-
price">${currency.format(product.price / 100)}</p></div>
        <p class="product-color">${product.color}</p>
        <button class="add-button" type="button" data-add="${product.id}" ${product.stock < 1 ?
'disabled' : ''}>${product.stock < 1 ? 'Sold out' : 'Add to bag'}</button>
```

```
      </div>
    </article>`).join('');
}

function cartEntries() {
  return Object.entries(cart).filter(([id, quantity]) => getProduct(id) && Number.isInteger(quantity) &&
quantity > 0);
}

function renderBag() {
  const items = cartEntries();
  const count = items.reduce((total, [, quantity]) => total + quantity, 0);
  const subtotal = items.reduce((total, [id, quantity]) => total + getProduct(id).price * quantity, 0);
  document.querySelector('#bag-count').textContent = count;
  document.querySelector('#drawer-count').textContent = `(${count})`;
  document.querySelector('#bag-subtotal').textContent = currency.format(subtotal / 100);
  bagItems.innerHTML = items.length ? items.map(([id, quantity]) => {
    const product = getProduct(id);
    return `<article class="bag-line">
      <img src="${product.image}" alt="" loading="lazy">
      <div class="bag-line-info"><h3>${product.name}</h3><p>${product.color}</p><div class="quantity-
control" aria-label="Quantity for ${product.name}">
        <button type="button" data-quantity="-1" data-id="${id}" aria-label="Remove one ${product.
name}">−</button>
        <span>${quantity}</span>
        <button type="button" data-quantity="1" data-id="${id}" aria-label="Add one ${product.name}" ${
quantity >= product.stock ? 'disabled' : ''}>+</button>
      </div></div><p class="bag-line-price">${currency.format(product.price * quantity / 100)}</p>
    </article>`;
  }).join('') : '<p class="empty-bag">Nothing in here just yet. Your next favorite is out there.</p>';
  checkoutButton.disabled = items.length === 0;
}

function openBag() {
  bagOverlay.hidden = false;
  bagDrawer.inert = false;
  bagDrawer.setAttribute('aria-hidden', 'false');
  requestAnimationFrame(() => bagDrawer.classList.add('is-open'));
  document.querySelector('#bag-close').focus();
}

function closeBag() {
  bagDrawer.classList.remove('is-open');
  bagDrawer.inert = true;
  bagDrawer.setAttribute('aria-hidden', 'true');
  window.setTimeout(() => { bagOverlay.hidden = true; }, 260);
  document.querySelector('#bag-open').focus();
}

function showToast(message) {
  const toast = document.querySelector('#toast');
  toast.textContent = message;
  toast.classList.add('is-visible');
  window.clearTimeout(toastTimer);
  toastTimer = window.setTimeout(() => toast.classList.remove('is-visible'), 2100);
}

document.addEventListener('click', (event) => {
  const addButton = event.target.closest('[data-add]');
  if (addButton) {
    const product = getProduct(addButton.dataset.add);
    if (product && (cart[product.id] || 0) < product.stock) {
      cart[product.id] = (cart[product.id] || 0) + 1;
      saveCart();
      renderBag();
      showToast(`${product.name} added to your bag`);
    }
  }

  const quantityButton = event.target.closest('[data-quantity]');
  if (quantityButton) {
    const id = quantityButton.dataset.id;
    cart[id] = Math.max(0, (cart[id] || 0) + Number(quantityButton.dataset.quantity));
    if (!cart[id]) delete cart[id];
    saveCart();
    renderBag();
  }

  const categoryLink = event.target.closest('[data-nav-category]');
  if (categoryLink) {
    event.preventDefault();
    selectedCategory = categoryLink.dataset.navCategory;
    document.querySelectorAll('[data-nav-category]').forEach((link) => link.classList.toggle('is-active',
 link === categoryLink));
    renderFilters();
    renderProducts();
    document.querySelector('#shop').scrollIntoView({ behavior: 'smooth' });
  }
});
```

```
searchInput.addEventListener('input', renderProducts);
sortSelect.addEventListener('change', renderProducts);
document.querySelector('#bag-open').addEventListener('click', openBag);
document.querySelector('#bag-close').addEventListener('click', closeBag);
bagOverlay.addEventListener('click', closeBag);
document.addEventListener('keydown', (event) => {
  if (event.key === 'Escape' && !bagOverlay.hidden) closeBag();
});

checkoutButton.addEventListener('click', async () => {
  checkoutButton.disabled = true;
  checkoutMessage.classList.remove('is-error');
  checkoutMessage.textContent = 'Placing your order...';
  try {
    const response = await fetch('/api/checkout', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ items: cartEntries().map(([product_id, quantity]) => ({ product_id,
quantity })) }),
    });
    const result = await response.json();
    if (!response.ok) throw new Error(result.detail || 'Checkout could not be completed.');
    cart = {};
    saveCart();
    renderBag();
    products = await fetchProducts();
    renderProducts();
    checkoutMessage.textContent = `Order ${result.order_id} confirmed. Total ${currency.format(result.
total / 100)}.`;
  } catch (error) {
    checkoutMessage.textContent = error.message;
    checkoutMessage.classList.add('is-error');
    renderBag();
  }
});

async function fetchProducts() {
  const response = await fetch('/api/products');
  if (!response.ok) throw new Error('The product collection is unavailable.');
  return response.json();
}

async function init() {
  try {
    products = await fetchProducts();
    renderFilters();
    renderProducts();
    renderBag();
    productGrid.setAttribute('aria-busy', 'false');
  } catch {
    productGrid.innerHTML = '<p class="empty-state">The shop is taking a little breather. Refresh to try
again.</p>';
    productGrid.setAttribute('aria-busy', 'false');
  }
}

init();
```

docs/requirements.txt

```
Markdown>=3.7,<4
reportlab>=4.2,<5
```

docs/build_guide.py

```
from html.parser import HTMLParser
from pathlib import Path
from xml.sax.saxutils import escape, quoteattr

import markdown
from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER, TA_LEFT
from reportlab.lib.pagesizes import letter
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import inch
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import (
    HRFlowable,
    KeepTogether,
    PageBreak,
    Paragraph,
    Preformatted,
    SimpleDocTemplate,
    Spacer,
    Table,
    TableStyle,
)


PROJECT_DIR = Path(__file__).resolve().parent.parent
DOCS_DIR = Path(__file__).resolve().parent
GUIDE_PATH = DOCS_DIR / "COMPLETE_GUIDE.md"
PDF_PATH = DOCS_DIR / "COMPLETE_GUIDE.pdf"

SOURCE_FILES = [
    ("main.py", "python"),
    ("requirements.txt", "text"),
    ("static/index.html", "html"),
    ("static/styles.css", "css"),
    ("static/app.js", "javascript"),
    ("docs/requirements.txt", "text"),
    ("docs/build_guide.py", "python"),
]


class Element:
    def __init__(self, tag, attrs=None):
        self.tag = tag
        self.attrs = dict(attrs or [])
        self.children = []


class MarkdownTreeParser(HTMLParser):
    VOID_TAGS = {"br", "hr", "img", "input", "meta", "link"}

    def __init__(self):
        super().__init__(convert_charrefs=True)
        self.root = Element("document")
        self.stack = [self.root]

    def handle_starttag(self, tag, attrs):
        element = Element(tag, attrs)
        self.stack[-1].children.append(element)
        if tag not in self.VOID_TAGS:
            self.stack.append(element)

    def handle_endtag(self, tag):
        for index in range(len(self.stack) - 1, 0, -1):
            if self.stack[index].tag == tag:
                del self.stack[index:]
                break

    def handle_data(self, data):
        self.stack[-1].children.append(data)


def plain_text(node):
    if isinstance(node, str):
        return node
    return "".join(plain_text(child) for child in node.children)


def inline_markup(node):
    if isinstance(node, str):
        return escape(node)

    tag = node.tag
    body = "".join(inline_markup(child) for child in node.children)
    if tag in {"strong", "b"}:
        return f"<b>{body}</b>"
```

```
    if tag in {"em", "i"}:
        return f"<i>{body}</i>"
    if tag == "code":
        return f'<font name="DejaVuSansMono" size="8">{body}</font>'
    if tag == "a" and "href" in node.attrs:
        return f"<link href={quoteattr(node.attrs['href'])}>{body}</link>"
    if tag == "br":
        return "<br/>"
    return body


def descendants(node, tag):
    return [child for child in node.children if isinstance(child, Element) and child.tag == tag]


def register_fonts():
    fonts = Path("/usr/share/fonts/dejavu")
    pdfmetrics.registerFont(TTFont("DejaVuSans", str(fonts / "DejaVuSans.ttf")))
    pdfmetrics.registerFont(TTFont("DejaVuSans-Bold", str(fonts / "DejaVuSans-Bold.ttf")))
    pdfmetrics.registerFont(TTFont("DejaVuSansMono", str(fonts / "DejaVuSansMono.ttf")))
    pdfmetrics.registerFont(TTFont("DejaVuSansMono-Bold", str(fonts / "DejaVuSansMono-Bold.ttf")))
    pdfmetrics.registerFontFamily(
        "DejaVuSans",
        normal="DejaVuSans",
        bold="DejaVuSans-Bold",
        italic="DejaVuSans",
        boldItalic="DejaVuSans-Bold",
    )


def make_styles():
    styles = getSampleStyleSheet()
    styles.add(ParagraphStyle(
        "GuideTitle", parent=styles["Title"], fontName="DejaVuSans-Bold",
        fontSize=24, leading=29, textColor=colors.HexColor("#262923"),
        alignment=TA_LEFT, spaceAfter=11,
    ))
    styles.add(ParagraphStyle(
        "GuideH1", parent=styles["Heading1"], fontName="DejaVuSans-Bold",
        fontSize=17, leading=21, textColor=colors.HexColor("#262923"),
        spaceBefore=14, spaceAfter=8, keepWithNext=True,
    ))
    styles.add(ParagraphStyle(
        "GuideH2", parent=styles["Heading2"], fontName="DejaVuSans-Bold",
        fontSize=12, leading=15, textColor=colors.HexColor("#397243"),
        spaceBefore=10, spaceAfter=5, keepWithNext=True,
    ))
    styles.add(ParagraphStyle(
        "GuideH3", parent=styles["Heading3"], fontName="DejaVuSans-Bold",
        fontSize=10, leading=13, textColor=colors.HexColor("#262923"),
        spaceBefore=7, spaceAfter=3, keepWithNext=True,
    ))
    styles.add(ParagraphStyle(
        "GuideBody", parent=styles["BodyText"], fontName="DejaVuSans",
        fontSize=8.7, leading=12, textColor=colors.HexColor("#32342e"),
        spaceAfter=6, splitLongWords=True,
    ))
    styles.add(ParagraphStyle(
        "GuideBullet", parent=styles["GuideBody"], leftIndent=15,
        firstLineIndent=-10, bulletIndent=0, spaceAfter=4,
    ))
    styles.add(ParagraphStyle(
        "GuideCode", fontName="DejaVuSansMono", fontSize=6.1, leading=7.6,
        leftIndent=5, rightIndent=5, borderColor=colors.HexColor("#e1e2db"),
        borderWidth=0.4, borderPadding=5, backColor=colors.HexColor("#f5f5f2"),
        spaceBefore=4, spaceAfter=8,
    ))
    styles.add(ParagraphStyle(
        "GuideTable", parent=styles["GuideBody"], fontSize=7.4,
        leading=9.3, spaceAfter=1,
    ))
    styles.add(ParagraphStyle(
        "GuideTableHeader", parent=styles["GuideTable"],
        fontName="DejaVuSans-Bold", textColor=colors.HexColor("#262923"),
    ))
    return styles


def add_table(element, story, styles, available_width):
    rows = []
    for row in element.children:
        if not isinstance(row, Element):
            continue
        if row.tag in {"thead", "tbody", "tfoot"}:
            row_elements = [child for child in row.children if isinstance(child, Element) and child.tag
== "tr"]
        elif row.tag == "tr":
            row_elements = [row]
        else:
```

```
            continue
        for row_element in row_elements:
            cells = [child for child in row_element.children if isinstance(child, Element) and child.tag
in {"th", "td"}]
            cell_style = styles["GuideTableHeader"] if cells and cells[0].tag == "th" else styles[
"GuideTable"]
            rows.append([Paragraph(inline_markup(cell), cell_style) for cell in cells])

    if not rows:
        return

    column_count = max(len(row) for row in rows)
    widths = [available_width / column_count] * column_count
    if column_count == 3:
        widths = [available_width * 0.34, available_width * 0.20, available_width * 0.46]
    table = Table(rows, colWidths=widths, repeatRows=1, hAlign="LEFT")
    table.setStyle(TableStyle([
        ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#eff0e8")),
        ("GRID", (0, 0), (-1, -1), 0.35, colors.HexColor("#ddded7")),
        ("VALIGN", (0, 0), (-1, -1), "TOP"),
        ("LEFTPADDING", (0, 0), (-1, -1), 5),
        ("RIGHTPADDING", (0, 0), (-1, -1), 5),
        ("TOPPADDING", (0, 0), (-1, -1), 4),
        ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
    ]))
    story.extend([table, Spacer(1, 7)])


def add_list(element, story, styles, depth=0):
    for item in descendants(element, "li"):
        children = [child for child in item.children if not (isinstance(child, Element) and child.tag in
{"ul", "ol"})]
        text = "".join(inline_markup(child) for child in children).strip()
        if text:
            style = ParagraphStyle(
                f"GuideBullet{depth}", parent=styles["GuideBullet"],
                leftIndent=15 + 10 * depth, firstLineIndent=-10,
            )
            bullet = "&#8226;" if element.tag == "ul" else f"{descendants(element, 'li').index(item) +
1}."
            story.append(Paragraph(f"{bullet}  {text}", style))
        for nested in item.children:
            if isinstance(nested, Element) and nested.tag in {"ul", "ol"}:
                add_list(nested, story, styles, depth + 1)


def build_story(root, styles, available_width):
    story = []
    for node in root.children:
        if isinstance(node, str):
            if node.strip():
                story.append(Paragraph(escape(node.strip()), styles["GuideBody"]))
            continue

        if node.tag in {"h1", "h2", "h3", "h4", "h5", "h6"}:
            title = plain_text(node)
            if node.tag == "h1" and title.startswith("Appendix:"):
                story.append(PageBreak())
            style_name = "GuideTitle" if node.tag == "h1" and not title.startswith("Appendix:") else {
                "h1": "GuideH1", "h2": "GuideH2", "h3": "GuideH3",
                "h4": "GuideH3", "h5": "GuideH3", "h6": "GuideH3",
            }[node.tag]
            story.append(Paragraph(inline_markup(node), styles[style_name]))
        elif node.tag == "p":
            story.append(Paragraph(inline_markup(node), styles["GuideBody"]))
        elif node.tag in {"ul", "ol"}:
            add_list(node, story, styles)
            story.append(Spacer(1, 3))
        elif node.tag == "pre":
            story.append(Preformatted(
                plain_text(node).rstrip("\n"), styles["GuideCode"], maxLineLength=105,
            ))
        elif node.tag == "table":
            add_table(node, story, styles, available_width)
        elif node.tag == "hr":
            story.append(HRFlowable(width="100%", thickness=0.6, color=colors.HexColor("#e1e2db"),
spaceBefore=5, spaceAfter=8))

    return story


def draw_footer(canvas, doc):
    canvas.saveState()
    canvas.setStrokeColor(colors.HexColor("#e1e2db"))
    canvas.setLineWidth(0.5)
    canvas.line(doc.leftMargin, 0.52 * inch, letter[0] - doc.rightMargin, 0.52 * inch)
    canvas.setFont("DejaVuSans", 7)
    canvas.setFillColor(colors.HexColor("#77786e"))
    canvas.drawString(doc.leftMargin, 0.35 * inch, "Form & Field | Complete Project Guide")
```

```
    canvas.drawRightString(letter[0] - doc.rightMargin, 0.35 * inch, f"Page {doc.page}")
    canvas.restoreState()


def main():
    guide = GUIDE_PATH.read_text(encoding="utf-8")
    sections = [guide, "\n# Appendix: Complete Source Files\n\nThese listings are generated directly
from the current project files.\n"]
    for relative_path, language in SOURCE_FILES:
        source = (PROJECT_DIR / relative_path).read_text(encoding="utf-8").rstrip()
        sections.extend([
            f"\n## `{relative_path}`\n\n",
            f"```{{.{language}}}\n{source}\n```\n",
        ])

    document_html = markdown.markdown(
        "".join(sections),
        extensions=["fenced_code", "sane_lists", "tables"],
    )
    parser = MarkdownTreeParser()
    parser.feed(document_html)

    register_fonts()
    doc = SimpleDocTemplate(
        str(PDF_PATH),
        pagesize=letter,
        rightMargin=0.72 * inch,
        leftMargin=0.72 * inch,
        topMargin=0.68 * inch,
        bottomMargin=0.72 * inch,
        title="Form & Field — Complete Project Guide",
        author="Form & Field Storefront Documentation",
    )
    styles = make_styles()
    available_width = letter[0] - doc.leftMargin - doc.rightMargin
    story = build_story(parser.root, styles, available_width)
    doc.build(story, onFirstPage=draw_footer, onLaterPages=draw_footer)

    print(f"Created {PDF_PATH}")


if __name__ == "__main__":
    main()
```